



Operating Systems

Unit 2

Subject Code: CI42

B.E Sem: IV

Credits : 4



Syllabus

Unit II

Process Management: Basic concept; Process scheduling; Operations on processes; Inter process Communication.

Threads: Overview; Multithreading models;

Process scheduling: Basic concepts, Scheduling criteria, scheduling algorithms, multiple processor scheduling, Algorithm evaluation.



The Evolution of Computing

- **Early Computers:** Allowed only *one program* to be executed at a time.
 - This single program had complete control of the system and access to all of the system's resources.
- **Contemporary Computer Systems:** Allow *multiple programs* to be loaded into memory and executed concurrently (at the same time).
- **The Need for Control:** Running multiple programs at once required firmer control and better compartmentalization of the various programs so they wouldn't interfere with each other.
- **The Solution:** These needs resulted in the invention of the **process**.



What is a Process?

- **Definition:** A process is defined simply as a **program in execution**.
- A process is the fundamental unit of work in any modern time-sharing computer system.
- **The Role of the Operating System:**
 - As operating systems become more complex, they are expected to do more on behalf of their users.
 - While the OS's main concern is executing user programs, it also needs to take care of various background system tasks that are better left outside the kernel itself.

- A computer system consists of a massive collection of processes:
 - **Operating-system processes:** Executing system code.
 - **User processes:** Executing user code.
- **Concurrent Execution:** Potentially, all of these system and user processes can execute concurrently.
- **CPU Multiplexing:** How does one CPU handle all of this? The CPU is *multiplexed* among them.
 - By rapidly switching the CPU between different processes, the operating system can make the computer highly productive.



The Process Concept & Terminology

- **What do we call CPU activities?** Different types of computer systems use different names for executing programs:
 - *Batch Systems*: Execute "**jobs**"
 - *Time-Shared Systems*: Execute "**user programs**" or "**tasks**"
- **The Unifying Term:** Because all of the activities—whether they are user tasks or OS background tasks—are fundamentally similar, we call all of them **processes**.
- **"Job" vs. "Process":** In operating system theory, the terms *job* and *process* are used almost interchangeably.

The Anatomy of a Process in Memory

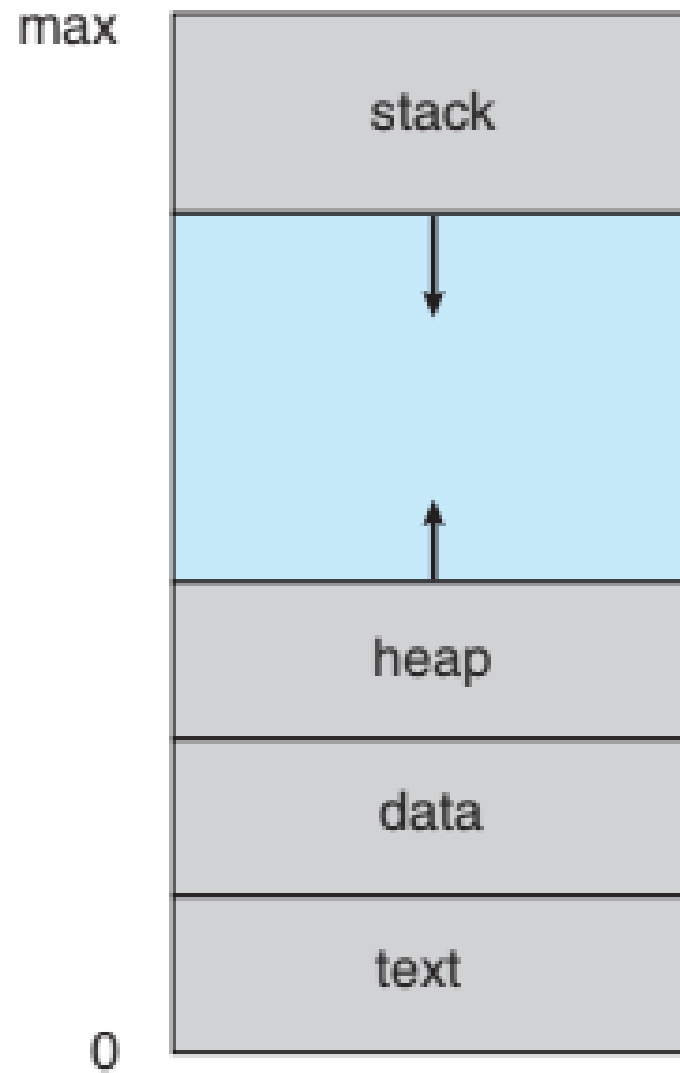


Figure 3.1 Process in memory.

The Anatomy of a Process in Memory

A process is much more than just the program code. It consists of several distinct parts in memory:

1. **Text Section:** The compiled program code itself.
2. **Current Activity:** Represented by the **Program Counter** (which tracks the next instruction to execute) and the contents of the processor's registers.
3. **Stack:** Contains temporary data, such as function parameters, return addresses, and local variables.
4. **Data Section:** Contains all the global variables used by the program.
5. **Heap:** Memory that is dynamically allocated to the process while it is running.



Passive vs. Active:

- A **program** is a *passive* entity—just a file containing a list of instructions stored on a disk (an executable file).
- A **process** is an *active* entity—it has a program counter and associated resources.

How it starts: A program becomes a process when the executable file is loaded into memory (e.g., by double-clicking an icon or typing the file name in a command line).

Multiple Instances:

- You can run the same program multiple times (e.g., opening several web browser windows).
- Each instance is a *completely separate process*.
- While they have the exact same **Text (code)** section, their **Data, Heap, and Stack** sections are entirely separate and unique.



Processes as Execution Environments

Meta-Processes: A process itself can serve as an execution environment for *other* code.

The Java Example: The Java Virtual Machine (JVM) is a perfect example of this concept.

- When you run a compiled Java program (e.g., typing `java Program.class`), the operating system does not run the Java code directly.
- Instead, the OS starts the **JVM as an ordinary process**.
- The JVM process then interprets the loaded Java code and takes actions on behalf of that code.

Simulation: This concept is essentially the same as running a simulation, where a program acts as a virtual machine executing a different instruction set.

Dynamic Nature of Processes: As a process executes, it changes state. The state is defined in part by the *current activity* of that process.

The Five Basic States:

1. **New:** The process is currently being created.
2. **Ready:** The process has everything it needs to run and is simply waiting to be assigned to a processor.
3. **Running:** Instructions are actively being executed by the CPU.
4. **Waiting (Blocked):** The process cannot continue yet; it is waiting for some event to occur (such as an I/O completion or receiving a signal).
5. **Terminated:** The process has completely finished its execution.

The Golden Rule of Processing: On any single processor (CPU core), **only one process can be running at any given instant**. However, **many** processes can be in the "Ready" and "Waiting" states at the same time.

Process States and Lifecycle

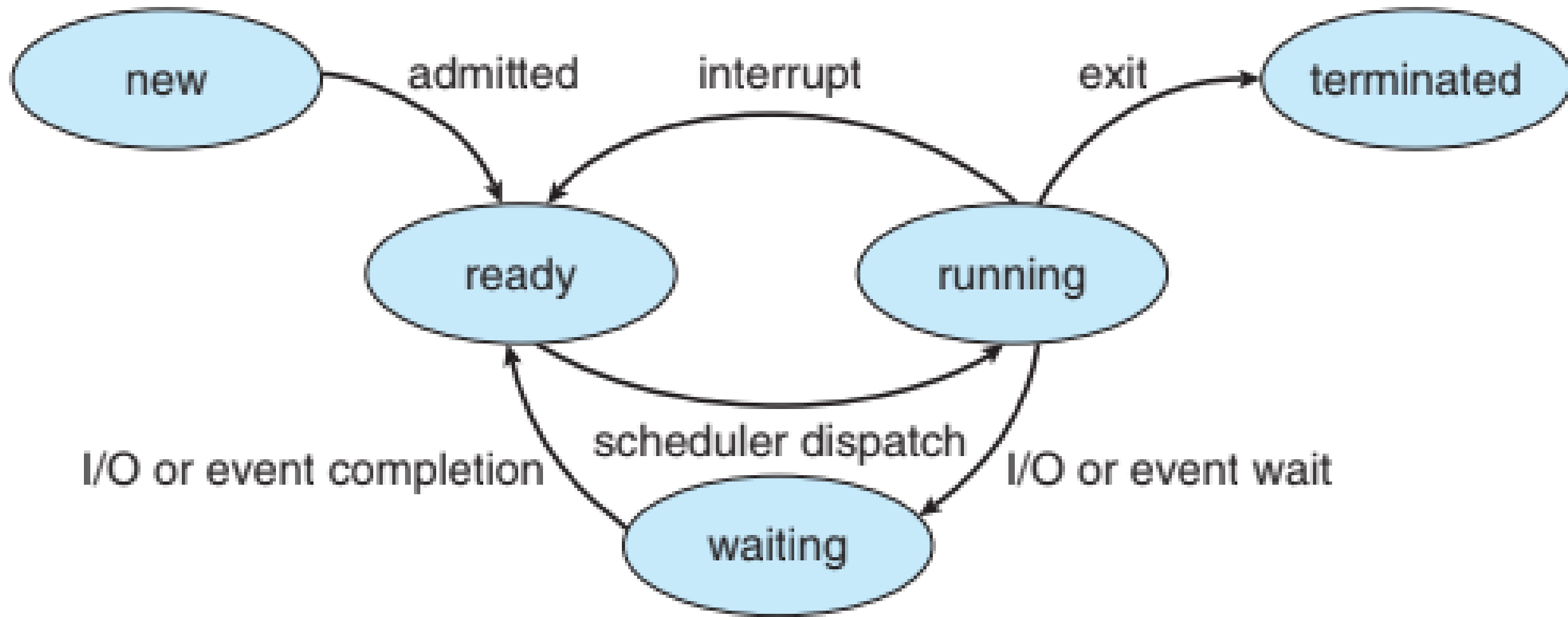


Figure 3.2 Diagram of process state.



What is it? Each process is represented in the operating system by a **Process Control Block (PCB)**, which is also called a task control block.

Purpose: The PCB serves as the central repository for any information that may vary from process to process.

Key Components of a PCB:

1. **Process state:** The current state (new, ready, running, waiting, halted, etc.).
2. **Program counter:** Indicates specific address of next instruction to be executed.
3. **CPU registers:** Includes accumulators, stack pointers, and general-purpose registers where temporary mathematical states are held.
4. **CPU-scheduling info:** Includes the process's priority and pointers to scheduling queues.
5. **Memory-management info:** Includes values of base/limit registers and page/segment tables.
6. **Accounting info:** Tracks the amount of CPU time used, time limits, and account/process numbers.
7. **I/O status info:** List of I/O devices allocated to process and list of open files.



Process Control Block

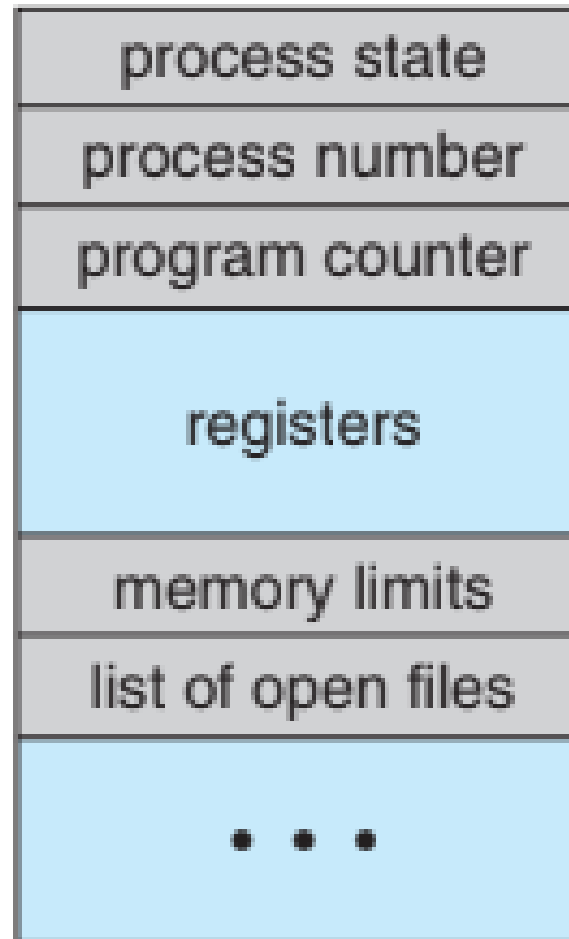


Figure 3.3 Process control block (PCB).



- **Handling Interrupts:** When an interrupt or system call occurs, the operating system must temporarily stop the running process.
- **State Save:** The system must save the current state information (specifically the program counter and CPU registers) into the process's PCB.
- **State Restore:** This saved information allows the OS to correctly restore and continue the process later without losing any data.
- **The Switch Mechanism:**
 1. Process P0 is executing.
 2. An interrupt occurs. Process P0's state is saved into **PCB0**.
 3. The CPU is briefly idle while the OS selects a new process.
 4. The state of Process P1 is reloaded from **PCB1**.
 5. Process P1 begins executing.



PCB in Action (CPU Switching)

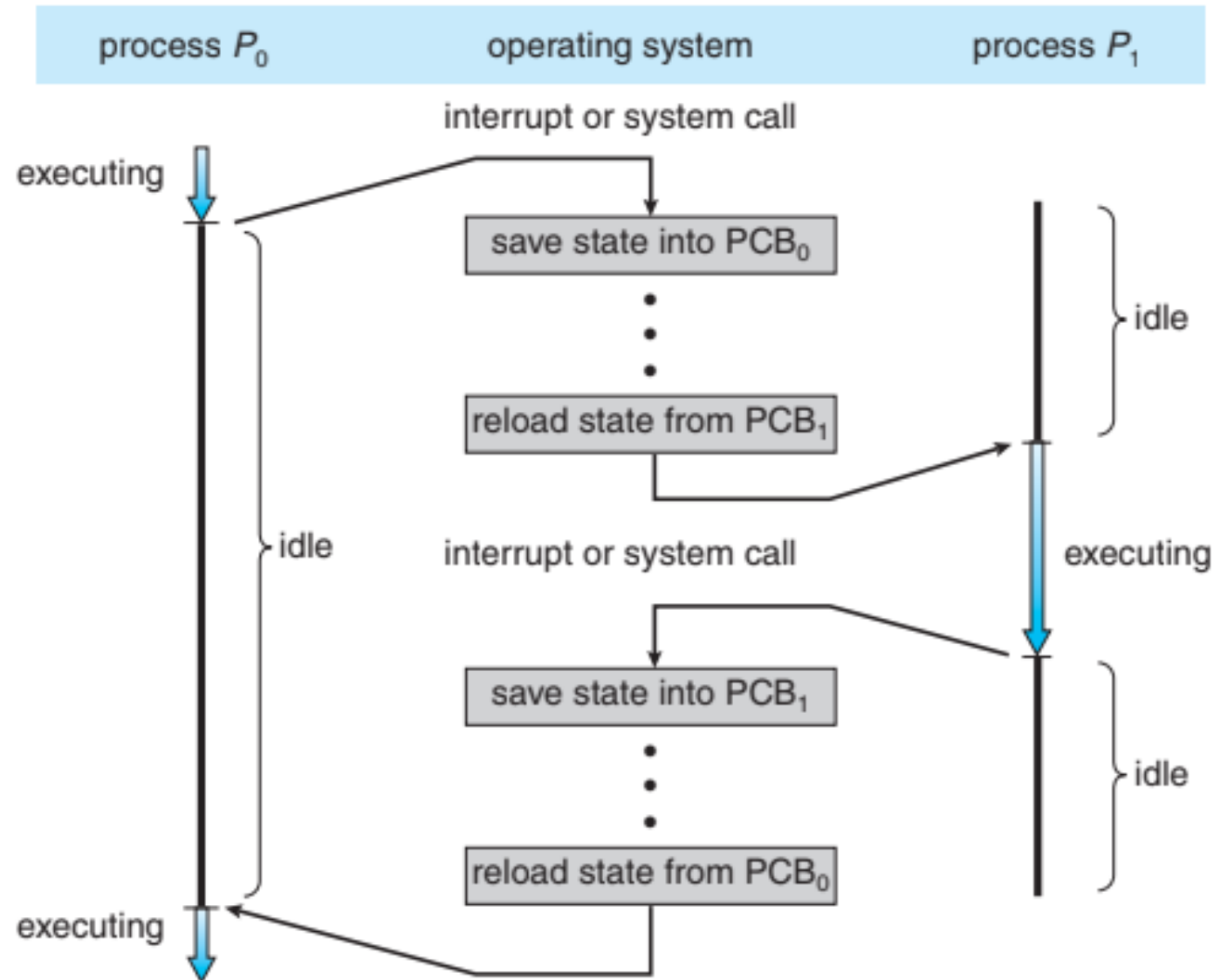


Figure 3.4 Diagram showing CPU switch from process to process.



The Goal of Multiprogramming: To have *some* process running at all times in order to **maximize CPU utilization**.

The Goal of Time Sharing: To switch the CPU among processes so frequently that users can interact with each program while it is running.

The Process Scheduler: The operating system component that meets these objectives by selecting an available process for program execution on the CPU.

Single-Processor Rule: For a single-processor system, there will *never* be more than one running process. If there are more, they must wait until the CPU is free.



As processes enter the system, they are organized into various waiting lists, called **queues**.

Job Queue: Consists of *all* processes currently in the system.

Ready Queue: Processes that are residing in main memory and are ready and waiting to execute.

- This is generally stored as a linked list.
- A ready-queue header contains pointers to the first and final Process Control Blocks (PCBs).

Device Queue: The list of processes waiting for a particular I/O device (like a busy hard disk). Each shared device has its own dedicated device queue.



Scheduling Queues

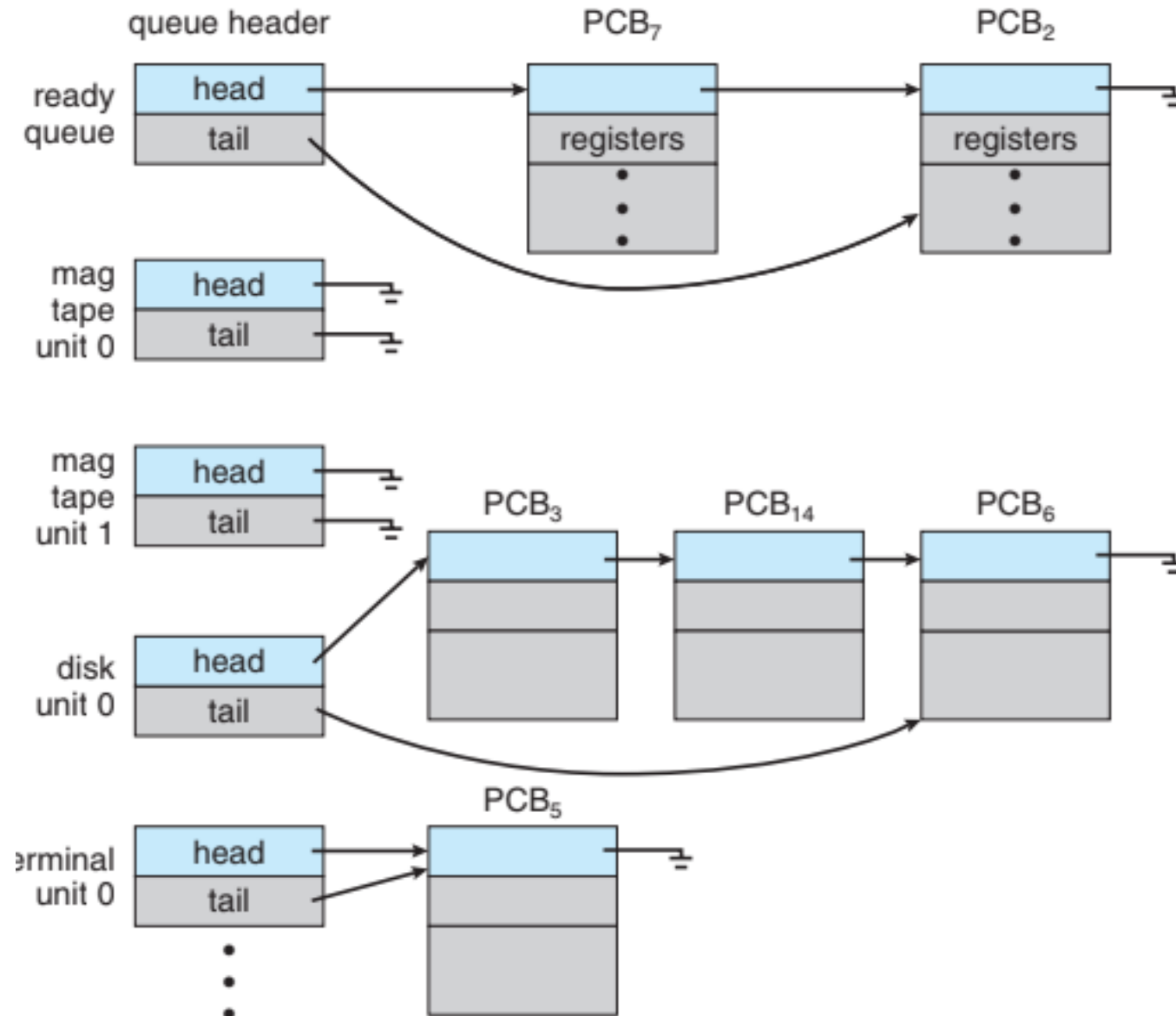


Figure 3.5 The ready queue and various I/O device queues.



Queueing Diagram: A common way to represent process scheduling. Rectangular boxes represent queues, circles represent resources, and arrows indicate the flow of processes.

The Execution Cycle:

1. A new process starts in the **ready queue**.
2. It is selected (dispatched) and allocated the CPU.
3. From the CPU, one of several events can occur:
 - It issues an **I/O request** → placed in an I/O queue.
 - It creates a **new child process** → waits for the child to terminate.
 - It is **forcibly interrupted** (time slice expires) → put back in the ready queue.

Termination: The process eventually cycles back to the ready queue until it terminates, at which point its PCB and resources are deallocated.



Process Flow and the Queueing Diagram

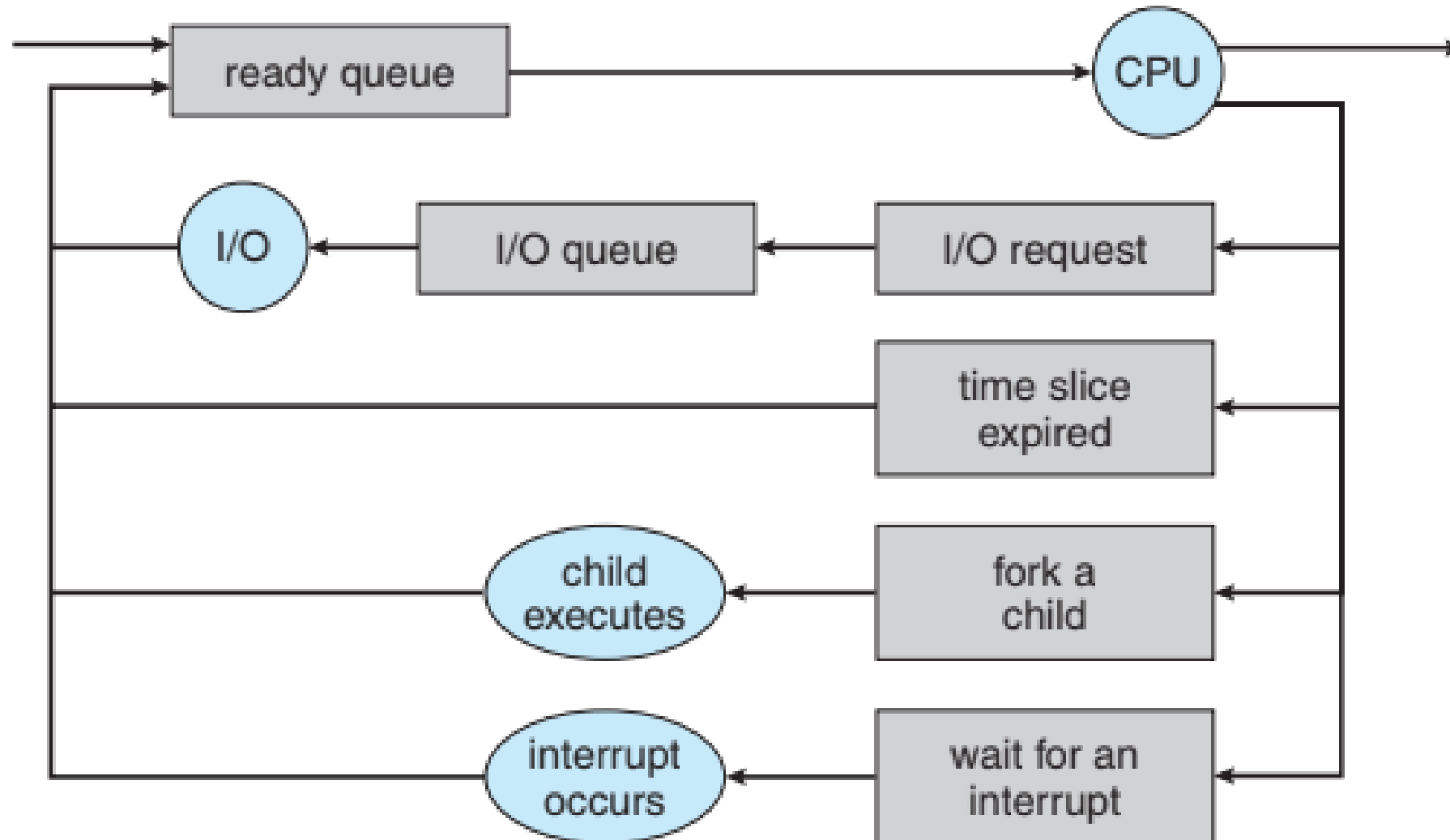


Figure 3.6 Queueing-diagram representation of process scheduling.



Types of Schedulers

As a process migrates among scheduling queues, the OS must select which processes run next using **schedulers**.

Long-Term Scheduler (Job Scheduler):

- Selects processes from a mass-storage pool (disk) and loads them into memory.
- Executes **infrequently** (minutes apart) and controls the *degree of multiprogramming* (the total number of processes in memory).
- Because it runs less frequently, it can afford to take more time to make careful scheduling decisions.



Short-Term Scheduler (CPU Scheduler):

- Selects from processes already in memory (ready queue) and allocates the CPU to one of them.
- Executes **very frequently** (e.g., at least once every 100 milliseconds).
- Because of its frequency, it must be incredibly **fast** to avoid wasting CPU time on overhead.



The Goal of the Long-Term Scheduler: To select a good "mix" of processes to keep the system balanced.

Two Types of Processes:

I/O-Bound Process: Spends more of its time doing I/O than doing computations.

CPU-Bound Process: Generates I/O requests infrequently, using more of its time doing heavy computations.

Why the Mix Matters:

If all processes are I/O bound → CPU ready queue is empty, short-term scheduler has nothing to do.

If all processes are CPU bound → I/O queues are empty, devices go unused.

Modern Systems: In time-sharing systems like UNIX and Windows, the long-term scheduler is often minimal or absent. They simply put every new process directly into memory for the short-term scheduler.



Medium-Term Scheduler & Swapping

The Problem: Sometimes an operating system becomes overcommitted, or the process mix needs to be improved.

The Solution: An intermediate level of scheduling called the **Medium-Term Scheduler**.

How it Works (Swapping):

It removes a process from memory (and from active contention for the CPU), thereby *reducing the degree of multiprogramming*.

The process is **swapped out** to a storage disk.

Later, the process is **swapped in** (reintroduced into memory), and its execution continues exactly where it left off.



Medium-Term Scheduler & Swapping

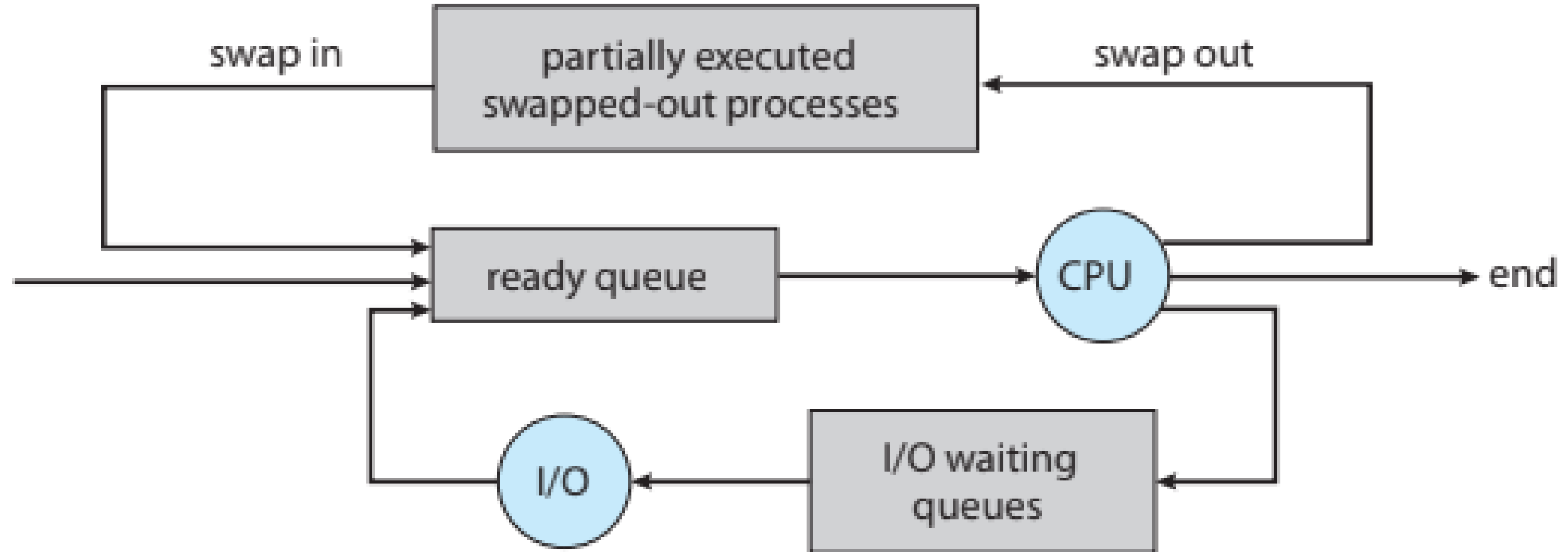


Figure 3.7 Addition of medium-term scheduling to the queueing diagram.



Parent and Child Processes: During execution, a process can create several new processes.

- The creator is the **parent process**.
- The new processes are the **children**.
- Because children can create their own children, this forms a massive **tree of processes**.

Process Identifier (PID): Most operating systems (UNIX, Linux, Windows) identify every single process using a unique integer called a **pid**. The OS uses this pid as an index to look up the process's attributes.



Resource Allocation & Initialization

Resource Needs: When a child process is created, it needs resources (CPU time, memory, files, I/O devices) to accomplish its task.

Where do resources come from?

The child obtains resources directly from the operating system.

The child is constrained to a *subset* of the parent process's resources.

Why restrict resources? Forcing a child to share or partition the parent's resources prevents any single process from overloading the system by maliciously creating too many greedy child processes.

Passing Data to the Child:

Passing Input: The parent can pass initialization data. (*Example: Parent gives the child the file name image.jpg, and the child must find and open it*).

Passing Resources: The parent can pass actual OS resources. (*Example: Parent opens image.jpg itself, and hands the already-open file directly to the child*).



The Linux Process Tree Example:

- The **init process (pid = 1)** serves as the ultimate root parent for all user processes.
- Once the system boots, init spawns tasks like web servers, print servers, or the login screen.
- *Command:* On UNIX/Linux, typing **ps -el** in the terminal will list complete information for all currently active processes.



Medium-Term Scheduler & Swapping

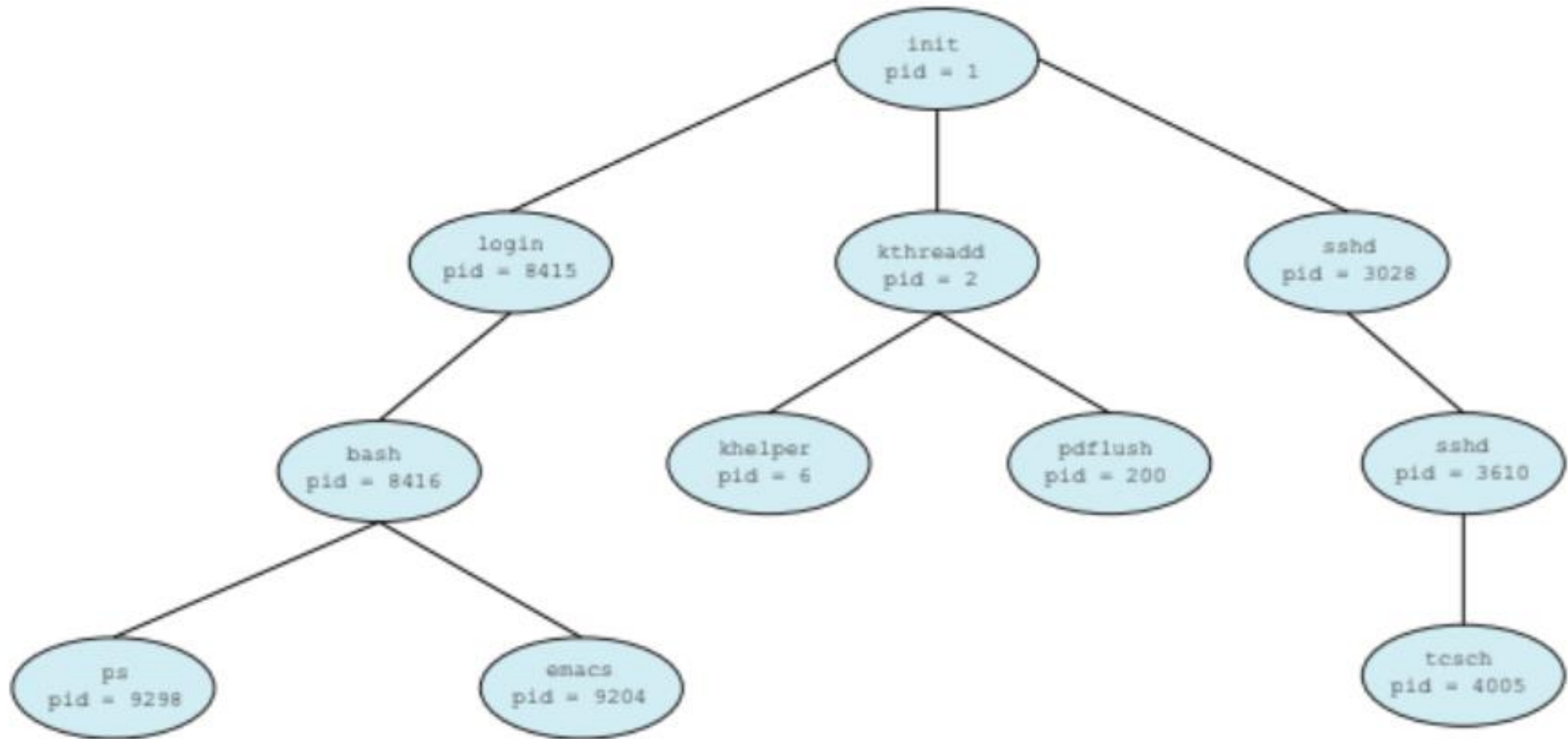


Figure 3.8 A tree of processes on a typical Linux system.

Execution Possibilities: When a process creates a new process, there are two main possibilities for how they execute:

- **Concurrent Execution:** The parent process continues to execute concurrently (at the same time) alongside its children.
- **Waiting:** The parent process waits (blocks) until some or all of its children have completely terminated before it continues.

Address-Space Possibilities: What does the memory of the new child process look like? There are two options:

- **Duplicate (Clone):** The child process is an exact duplicate of the parent process (it has the exact same program code and data as the parent).
- **New Program:** The child process has a completely new, different program loaded into it immediately.



The fork() System Call: In UNIX, a new process is created by the fork() system call. The new child consists of an exact copy of the address space of the parent.

Return Codes: Both parent and child continue execution at the instruction immediately after the fork().

- Child receives a return code of **zero**.
- Parent receives a return code containing the **process identifier (PID) of the child**.

The exec() System Call: Often used after a fork(). It loads a new binary file into memory, completely destroying the memory image of the old program, and starts its execution.

The wait() System Call: The parent can issue a wait() command to move itself off the ready queue until the child terminates.

Process Creation in UNIX

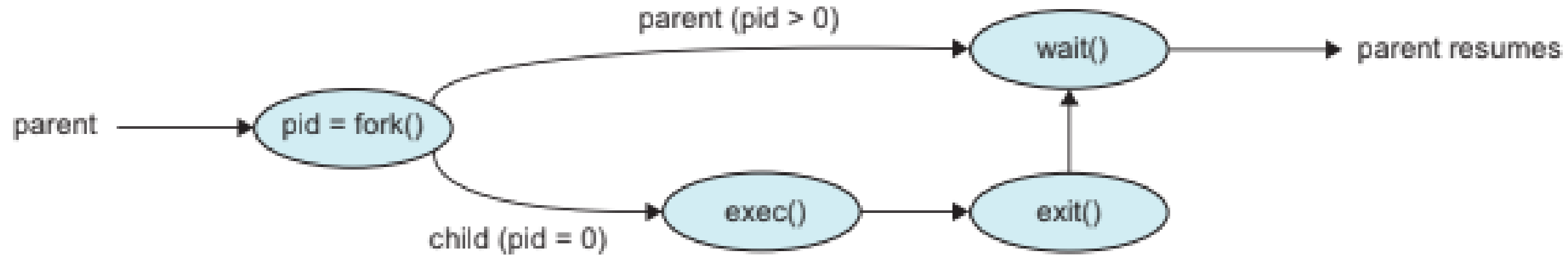


Figure 3.10 Process creation using the `fork()` system call.



Process Creation in Windows

- Unlike the "copy-paste" nature of UNIX fork(), Windows uses a **direct creation** approach.
- There is no cloning; a process is born as a new entity with its own executable right from the start.

Key Differences: Windows vs. UNIX

- **No Cloning:** CreateProcess() loads a specific program into the child's address space immediately.
- **Parameter Complexity:** * UNIX fork() = 0 parameters.
- Windows CreateProcess() = 10 parameters.



The Anatomy of CreateProcess()

- **Application & Command Line:** Defines *what* to run
- **Security & Inheritance:** Middle parameters handle process/thread handles and environment blocks.
- **The Structures:** The final two parameters are pointers to:
 - STARTUPINFO: Window appearance and handles.
 - PROCESS_INFORMATION: Receives identification handles for the new child.

Waiting for the Child

- To synchronize, Windows uses WaitForSingleObject() instead of wait().
- **Mechanism:** The parent uses the hProcess handle from the PROCESS_INFORMATION structure.
- **Result:** The parent "pauses" until the child signals it has finished.



Process Termination



Normal Termination: A process terminates when it finishes executing its final statement and asks the operating system to delete it using the `exit()` system call.

- The process returns a status value (an integer) to its parent.
- The OS completely deallocates all resources (physical/virtual memory, open files, I/O buffers).

Forced Termination: A parent process can deliberately terminate a child process (e.g., using `TerminateProcess()` in Windows).



Reasons a Parent Terminates a Child:

1. The child has exceeded its usage of allocated resources.
2. The task assigned to the child is no longer required.
3. The parent is exiting, and the OS does not allow a child to continue without a parent.

Cascading Termination: A phenomenon initiated by the OS in certain systems: if a parent process terminates (normally or abnormally), then all of its children must also be terminated.



The wait() System Call: A parent process uses wait() to pause its own execution until a child terminates, allowing the parent to collect the child's exit status and PID.

Zombie Process: A process that has terminated, but its parent has not yet called wait().

- Its resources are deallocated, but its process table entry remains to hold the exit status.
- All processes become zombies briefly. The entry is released once the parent finally calls wait().



Orphan Process: A child process whose parent has terminated without invoking `wait()`.

Handling Orphans (Linux/UNIX):

- The OS solves this by assigning the `init` process (the root of the process tree, PID 1) as the new parent to the orphan.
- The `init` process periodically calls `wait()` to collect exit statuses and clean up the process table.



Interprocess Communication



Independent vs. Cooperating Processes

Processes executing concurrently can be categorized into two types:

- **Independent Processes:** Cannot affect or be affected by other processes. They do not share data with any other process.
- **Cooperating Processes:** Can affect or be affected by other processes. Any process that shares data with other processes is cooperating.



Reasons for Process Cooperation

Why do we need processes to communicate?

- **Information sharing:** Multiple users or tasks may be interested in the same piece of information (e.g., concurrent access to a shared file).
- **Computation speedup:** A task can be broken into subtasks that execute in parallel (requires a computer with multiple processing cores).
- **Modularity:** A system can be constructed in a modular fashion, dividing functions into separate processes or threads.
- **Convenience:** Even a single user might want to edit a document, listen to music, and compile code all at same time.



Cooperating processes require an Interprocess Communication (IPC) mechanism to exchange data.

There are two fundamental IPC models:

- 1. Shared Memory**
- 2. Message Passing**



Shared Memory:

- A region of memory shared by processes is established.
- System calls are only required to establish the region; afterward, routine memory accesses are used (no kernel assistance).
- Advantage: Generally faster than message passing.
- Disadvantage: On modern multicore systems, it can suffer from cache coherency issues as shared data migrates among different core caches.



Message Passing:

- Communication takes place via messages exchanged between processes.
- Advantage: Useful for exchanging smaller amounts of data (no conflicts to avoid) and easier to implement in distributed systems.
- Disadvantage: Typically implemented using system calls, requiring time-consuming kernel intervention for every message.

Interprocess Communication (IPC) Models

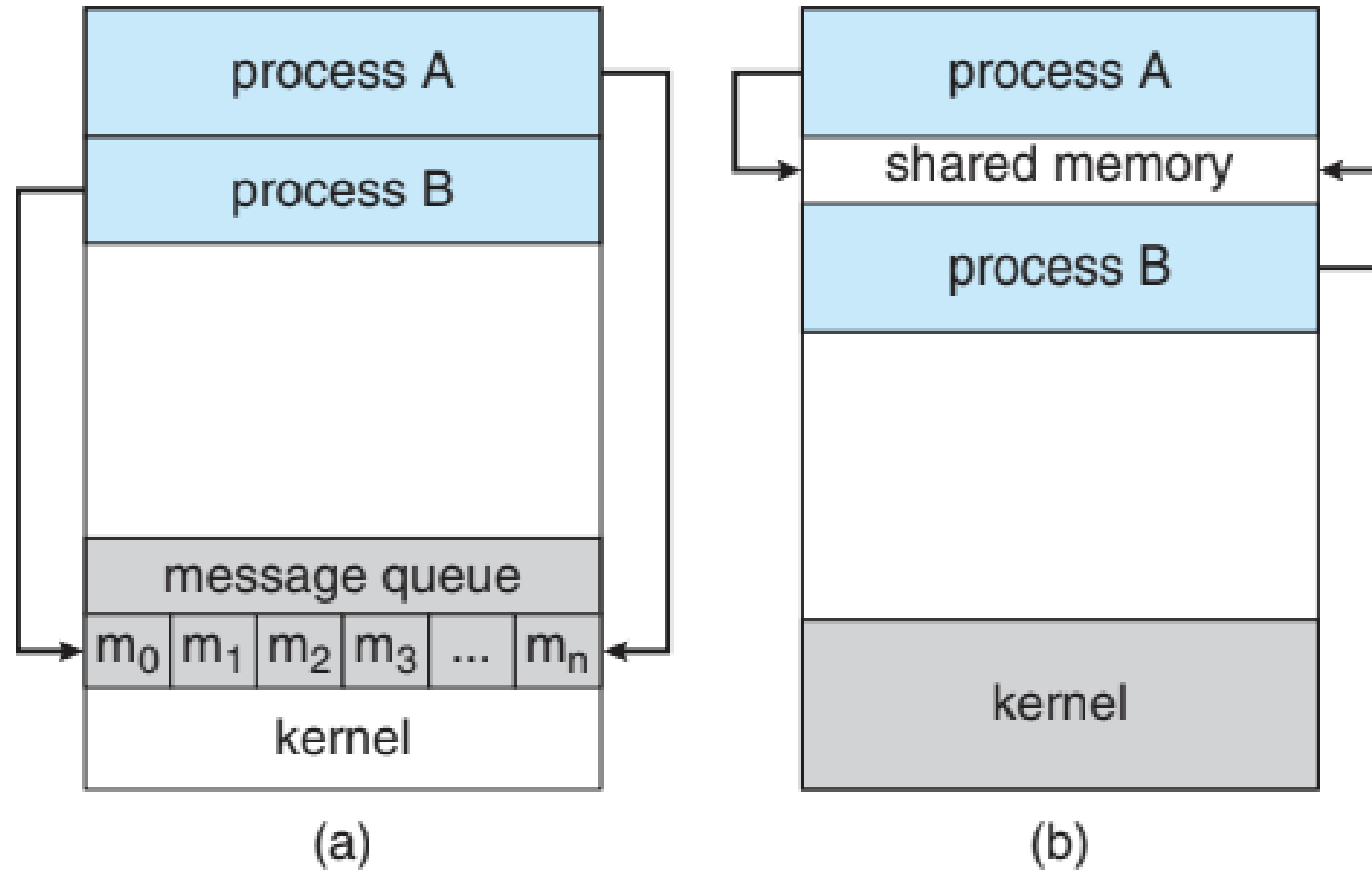


Figure 3.12 Communications models. (a) Message passing. (b) Shared memory.



Interprocess Communication (IPC) Models

Feature	Shared Memory	Message Passing
Mechanism	Processes share a common region of physical memory.	Processes communicate by sending and receiving packets of data.
Kernel Involvement	Only required to establish the memory region. Subsequent access is direct.	Required for every exchange (via system calls like send() and receive()).
Speed	Faster; data transfer occurs at memory speeds without overhead.	Slower; incurs kernel intervention and context switching overhead.
Data Volume	Ideal for exchanging large amounts of data.	Best for smaller amounts of data.
Ease of Use	Harder to manage; requires manual synchronization (e.g., semaphores) to avoid conflicts.	Easier to implement; synchronization is often handled by the messaging protocol.
Environment	Best for local systems (single machine).	Essential for distributed systems (across a network).
Modern Issue	Cache Coherency: Shared data migrating between different core caches can cause performance lags.	Overhead: Frequent system calls can create a bottleneck for high-frequency messaging.



Shared-Memory Systems: How It Works

The OS Memory Rule: Normally, the operating system strictly prevents one process from accessing another process's memory to ensure security and stability.

Breaking the Rule: Shared memory requires that two or more processes explicitly agree to remove this restriction.

How it is Established:

- A shared-memory region is created, typically residing in the address space of the creating process.
- Other cooperating processes that wish to communicate must explicitly attach this shared segment to their own address spaces.



The Producer-Consumer Problem

A Classic Paradigm: The producer-consumer problem is a common metaphor for cooperating processes.

The Roles:

- **Producer:** A process that creates information/data. (Example: A compiler producing assembly code, or a Web Server producing HTML files,).
- **Consumer:** A process that consumes the information/data. (Example: An assembler consuming assembly code, or a Client Web Browser reading the HTML files,).



The Producer-Consumer Problem

The Buffer: To allow both to run concurrently, there must be a shared "buffer" (waiting area) to hold items.

Two Types of Buffers:

- **Unbounded Buffer:** Places no practical limit on the size of the buffer. The producer can always produce, but the consumer may have to wait if it is empty.
- **Bounded Buffer:** Has a fixed, strictly limited size. The consumer waits if it is empty, and the producer must wait if the buffer is entirely full.



Implementing a Bounded Buffer

The Data Structure: A shared bounded buffer is implemented in code as a circular array with two logical pointers.

The Pointers:

- **in:** Points to the next free (empty) position in the buffer.
- **out:** Points to the first full (occupied) position in the buffer.

State Conditions: Because it is circular, we use the modulo operator (%) to wrap around the end of the array back to the beginning.

- **Buffer is Empty:** when $in == out$.
- **Buffer is Full:** when $((in + 1) \% BUFFER_SIZE) == out$.



Implementing a Bounded Buffer

```
#define BUFFER_SIZE 10

typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```



The producer process using shared memory

item next_produced; 3.4 Interprocess Communic

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (((in + 1) % BUFFER_SIZE) == out)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
}
```



The consumer process using shared memory

```
item next_consumed;

while (true) {
    while (in == out)
        ; /* do nothing */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_consumed */
}
```




Message-Passing Systems



The Concept: An IPC mechanism that allows processes to communicate and synchronize their actions without sharing the same address space.

Best Use Case: Particularly useful in a distributed environment where processes reside on entirely different computers connected by a network.

- **Example:** An Internet chat program where participants exchange messages across the globe.



Fundamental Operations: The OS message-passing facility provides at least two basic operations:

- `send(message)`
- `receive(message)`

The Message Size Tradeoff:

- **Fixed-sized messages:** The system-level implementation for the OS is straightforward, but it makes the task of programming much more difficult.
- **Variable-sized messages:** The programming task becomes simpler, but it requires a much more complex system-level implementation by the OS.



The Communication Link: If processes P and Q want to communicate, they must send and receive messages from each other. Therefore, a communication link must exist between them.

Physical vs. Logical: We are not concerned with the physical hardware link (like a shared bus or network cable), but rather the logical implementation of the link by the OS.



Implementation of Communication Links

Three Major Design Choices: When OS designers create a send()/receive() facility, they must choose how to implement the link:

- 1. Naming**
- 2. Synchronization**
- 3. Buffering**



Naming & Direct Communication

Naming Problem: For processes to communicate, they must have a specific way to refer to each other. Operating systems handle this using either direct or indirect communication.

Direct Communication: Each process must explicitly name the recipient or sender of the communication.

Link Properties in Direct Communication:

- A link is established automatically between any pair of processes that want to communicate; they only need to know each other's identity.
- A link is associated with exactly two processes.
- Between each pair, there exists exactly one link.



Symmetric vs. Asymmetric Addressing:

- Symmetric: Both sender and receiver must explicitly name each other (`send(P, message)` and `receive(Q, message)`).
- Asymmetric: Only the sender names the recipient (`send(P, message)`). The recipient accepts messages from anyone (`receive(id, message)`), and the system sets the variable `id` to the sender's name.

The Major Disadvantage: This scheme severely limits modularity. Because process identifiers are hard-coded, changing a single process's ID requires developers to find and modify every single reference to it across all other process definitions.



The Mailbox Concept: With indirect communication, messages are sent to and received from mailboxes (or ports).

- A mailbox is viewed abstractly as an object where processes can place and remove messages.
- Each mailbox has a unique identification (e.g., POSIX message queues use an integer value to identify a mailbox).

The Primitives: Two processes can communicate only if they share a mailbox.

- `send(A, message)`: Send a message to mailbox A.
- `receive(A, message)`: Receive a message from mailbox A.



Properties of the Communication Link:

- A link is established only if both processes share the mailbox.
- A link may be associated with more than two processes.
- Between any pair of processes, multiple different links may exist, each corresponding to a different mailbox.



The Multiple-Receiver Problem

The Scenario: Suppose processes P1, P2, and P3 all share mailbox A.

- Process P1 sends a message to Mailbox A.
- Both P2 and P3 execute a receive() from Mailbox A at the exact same time.
- Question: Which process actually gets the message?



The Multiple-Receiver Problem

The OS Solutions: The answer depends on which method the operating system designers choose to implement:

- **Limit Links:** Allow a link to be associated with two processes at most (preventing the scenario entirely).
- **Enforce Turns:** Allow at most one process at a time to successfully execute a receive() operation.
- **System Selection:** Allow the OS to arbitrarily select who gets the message (e.g., using a round-robin algorithm where processes take turns). The system may then notify the sender about who received it.



Mailbox Ownership

Process-Owned Mailboxes: The mailbox is part of the address space of a specific process.

Roles: The owner can only receive messages; the user can only send messages.

Lifecycle: There is no confusion about who receives messages. When the owner process terminates, the mailbox disappears completely. Any process that tries to send a message to it must be notified that it no longer exists.



Mailbox Ownership

OS-Owned Mailboxes: The mailbox has an existence of its own and is not attached to any particular process.

OS Mechanisms: The OS must provide tools to create, send/receive, and delete the mailbox.

Ownership: The process that creates it is the default owner (and the only receiver). However, ownership and receiving privileges can be passed to other processes via system calls, allowing for multiple receivers over time.



The Primitives: Communication between processes always takes place through calls to `send()` and `receive()` primitives.

Design Options: Operating systems can implement these primitives in two different ways, known generally as synchronous or asynchronous behavior.



Message Passing Behaviors:

- **Blocking send (Synchronous):** The sending process is blocked (paused) until the message is safely received by the receiving process or by the mailbox.
- **Nonblocking send (Asynchronous):** The sending process fires off the message and immediately resumes its operation without waiting.
- **Blocking receive (Synchronous):** The receiver blocks (waits) until a message is actually available to be picked up.
- **Nonblocking receive (Asynchronous):** The receiver attempts to get a message and immediately retrieves either a valid message or a null (indicating the mailbox is empty).

The Rendezvous & Producer-Consumer Solution

Combinations: Different combinations of blocking and nonblocking send() and receive() are possible.

The Rendezvous: When both the send() and the receive() operations are configured to be blocking, we have a rendezvous between the sender and the receiver.

Simplifying Producer-Consumer:

- Using blocking statements makes the classic producer-consumer problem entirely trivial to solve.
- The Producer: Merely invokes a blocking send() call and simply waits until the message is delivered.
- The Consumer: Invokes a blocking receive() call and naturally blocks until a new message is available from the producer.



Message Buffering

The Temporary Queue: Whether processes use direct or indirect communication, the messages they exchange must reside in a temporary queue while in transit.

Queue Implementation Types: Operating systems implement these queues in one of three ways:

- 1. Zero Capacity (No Buffering):** The queue has a maximum length of exactly zero.
 - The link cannot hold any waiting messages.
 - Behavior: The sender must block and wait until the recipient actively receives the message.

Message Buffering

2. **Bounded Capacity (Automatic Buffering):** The queue has a finite length (e.g., n messages).
 - Behavior: As long as the queue isn't full, the sender drops the message and continues execution without waiting.
 - If the link is completely full, the sender must block until space becomes available.
3. **Unbounded Capacity (Automatic Buffering):** The queue's length is potentially infinite.
 - Behavior: Any number of messages can wait in it. The sender never blocks.